RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Department of Computer Science and Engineering

CS23334 Fundamentals of Data Science Lab

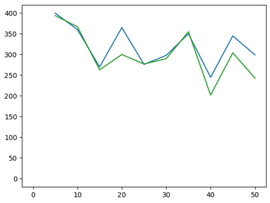
III semester II Year (2023R)

Name of the Student : Yashwanth Kumar V

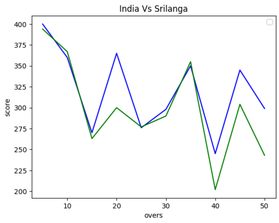
Register Number : 240701609

```
#day-1
#yashwanth kumar v
#240701609
#Matplotlib
#Line plot
```

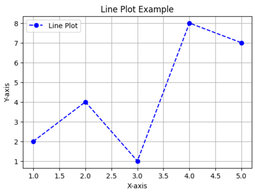
```
import matplotlib.pyplot as plt
overs=list(range(5,51,5))
India_Score=[400,360,270,365,276,298,350,245,345,299]
Srilanga_Score=[394,367,263,300,277,290,355,282,304,243]
plt.plot(overs,India_Score,'color'=='blue')
plt.plot(overs,Srilanga_Score)
plt.show()
plt.title("India Vs Srilanga")
plt.xlabel("overs")
plt.ylabel("score")
plt.legend()
plt.plot(overs,India_Score,color="blue",label="India")
plt.plot(overs,Srilanga_Score,color="green",label="Srilanga")
```



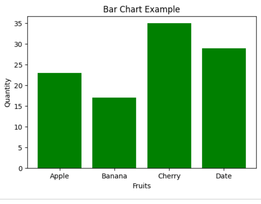
```
/tmp/ipython-input-1389799779.py:11: UserWarning: No artists with labels found to put in legend.  Note that artists whose label start with an underscore are ignored when legend
  plt.legend()
[<matplotlib.lines.Line2D at 0x7eda6d5207d0>]
```



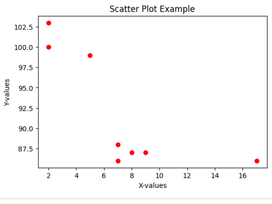

```
#Line plot
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 8, 7]
plt.figure(figsize=(6, 4))
plt.plot(x, y, color='blue', marker='o', linestyle='--', label='Line Plot')
plt.title("Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()
```



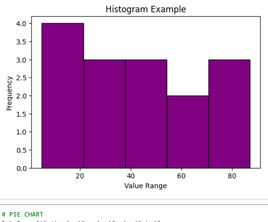

```
# BAR CHART
categories = ['Apple', 'Banana', 'Cherry', 'Date']
values = [23, 17, 35, 29]

plt.figure(figsize=(6, 4))
plt.bar(categories, values, color='green')
plt.title("Bar Chart Example")
plt.xlabel("Fruits")
plt.ylabel("Quantity")
plt.show()
```



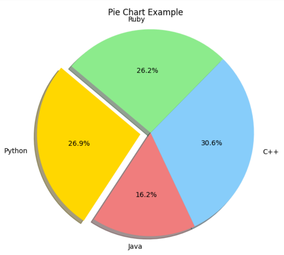

```
# SCATTER PLOT
x_scatter = [5, 7, 8, 7, 2, 17, 2, 9]
y_scatter = [99, 86, 87, 88, 100, 86, 103, 87]
plt.figure(figsize=(6, 4))
plt.scatter(x_scatter, y_scatter, color='red')
plt.title("Scatter Plot Example")
plt.xlabel("X-values")
plt.ylabel("Y-values")
plt.show()
```




```
# HISTOGRAM
data = [22, 87, 5, 43, 56, 73, 55, 54, 11, 20, 51, 5, 79, 31, 27]
plt.figure(figsize=(6, 4))
plt.hist(data, bins=5, color='purple', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.show()
```




```
# PIE CHART
labels = ['Python', 'Java', 'C++', 'Ruby']
sizes = [215, 130, 245, 210]
colors = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen']
explode = (0.1, 0, 0, 0)
plt.figure(figsize=(6, 6))
plt.pie(sizes, explode=explode, labels=labels, colors=colors,autopct='%1.1f%%', shadow=True, startangle=140)
plt.title("Pie Chart Example")
plt.axis('equal')
plt.show()
```



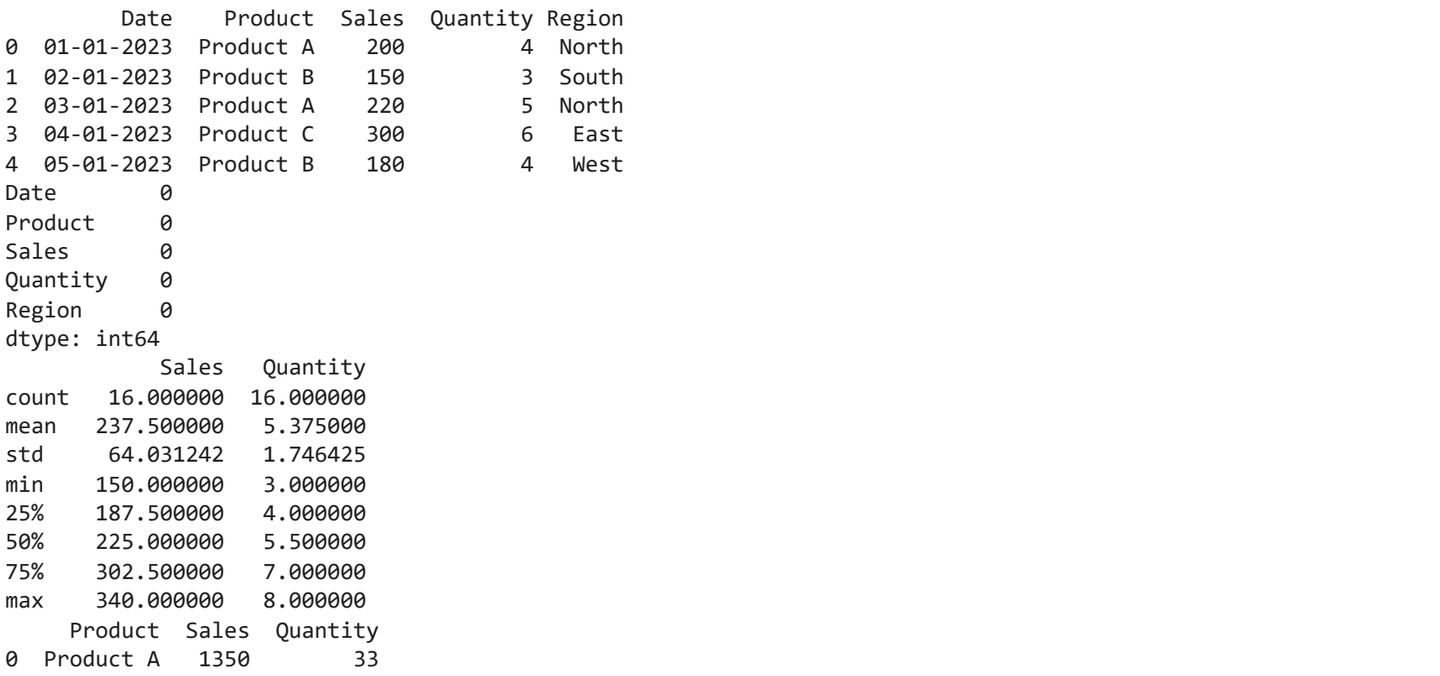
[ ]

```
#Exp No:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.
#yashwanth kumar v
#240701609
#experiment-2
```

[5] ✓ 0s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

file_path = '/content/sales_data.csv'
df = pd.read_csv(file_path)

print(df.head())
print(df.isnull().sum())

df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print(df.describe())

product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print(product_summary)

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y', errors='coerce')

sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
                             aggfunc=np.sum, fill_value=0)
print(pivot_table)

correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
/tmp/ipython-input-2753605132.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the op


  df['Sales'].fillna(df['Sales'].mean(), inplace=True)
```



Commands + Code + Text Run all

RAM Disk

```
#Experiment(Handling Missing and Inappropriate Data in a Dataset)
#yashwanth kumar v
#240701609
```

[1]

```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/Hotel_Dataset.csv")
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

Next steps: Generate code with df New interactive sheet

[2]

```
df.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	True
10	False

dtype: bool

[3]

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
```

[4]

```
df.drop_duplicates(inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

Next steps: Generate code with df New interactive sheet

[5]

```
len(df)
```

```
10
```

[6]

```
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

[7]

```
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

Next steps: Generate code with df New interactive sheet

[8]

```
df.drop(['Age_Group.1'],axis=1,inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777

Next steps: Generate code with df New interactive sheet

[9]

```
df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
df
```

```
/tmp/ipython-input-2080958306.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0)
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df.CustomerID.loc[df.CustomerID<0]=np.nan
/tmp/ipython-input-2080958306.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.CustomerID.loc[df.CustomerID<0]=np.nan
/tmp/ipython-input-2080958306.py:2: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0)
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df.Bill.loc[df.Bill<0]=np.nan
/tmp/ipython-input-2080958306.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.Bill.loc[df.Bill<0]=np.nan
/tmp/ipython-input-2080958306.py:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0)
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
/tmp/ipython-input-2080958306.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	veg	1300.0	2	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2	120000.0

Next steps: Generate code with df | New interactive sheet

[10]

```
df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
df
```

```
/tmp/ipython-input-2129877948.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) t
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
/tmp/ipython-input-2129877948.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	veg	1300.0	2.0	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Vegetarian	989.0	2.0	45000.0
5	6.0	35+	3	Ibys	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Vegetarian	1000.0	NaN	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	NaN	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	NaN
9	10.0	30-35	5	RedFox	non-Veg	NaN	4.0	87777.0

Next steps: Generate code with df | New interactive sheet

[11]

```
df.Age_Group.unique()
```

```
array(['20-25', '30-35', '25-30', '35+'], dtype=object)
```

[12]

```
df.Hotel.unique()
```

```
array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)
```

[13]

```
df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
df.FoodPreference.unique
```

```
/tmp/ipython-input-456600217.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
```

pandas.core.series.Series.unique
def unique() -> ArrayLike

```
Notes
-----
Returns the unique values as a NumPy array. In case of an
extension-array backed Series, a new
:class:`~api.extensions.ExtensionArray` of that type with just
the unique values is returned. This includes
```

[14]

```
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-Veg'],'Non-Veg',inplace=True)
```

[15]

```
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
df
```

```
/tmp/ipython-input-3711388855.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
/tmp/ipython-input-3711388855.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
/tmp/ipython-input-3711388855.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
/tmp/ipython-input-3711388855.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.Bill.fillna(round(df.Bill.mean()),inplace=True)
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	Veg	1300.0	2.0	40000.0

1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5	RedFox	Non-Veg	1801.0	4.0	87777.0

Next steps: Generate code with df New interactive sheet

[ ]

Start coding or generate with AI.

Variables Terminal 5:49 PM Python 3

Commands + Code + Text ▷ Run all

RAM Disk

```
#experiment-4(Experiment to understand the data preprocessing in Data science)
#yashwanth kumar v
#240701609
```

```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/pre_process_datasample.csv")
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
df.Country.mode()
```

	Country
0	France

dtype: object

```
df.Country.mode()[0]
```

```
'France'
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
df
```

```
/tmp/ipython-input-1020198583.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.Country.fillna(df.Country.mode()[0],inplace=True)
/tmp/ipython-input-1020198583.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.Age.fillna(df.Age.median(),inplace=True)
/tmp/ipython-input-1020198583.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the oper


  df.Salary.fillna(round(df.Salary.mean()),inplace=True)
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	63778.0	Yes
5	France	35.0	58000.0	Yes
6	Spain	38.0	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

Next steps: Generate code with df | New interactive sheet

```
pd.get_dummies(df.Country)
```

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	False	True	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	True	False	False
8	False	True	False
9	True	False	False

```
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],axis=1)
updated_dataset
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	False	True	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	63778.0	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	38.0	52000.0	No
7	True	False	False	48.0	79000.0	Yes
8	False	True	False	50.0	83000.0	No
9	True	False	False	37.0	67000.0	Yes

Next steps: Generate code with updated_dataset | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
updated_dataset
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	0
1	False	False	True	27.0	48000.0	1
2	False	True	False	30.0	54000.0	0
3	False	False	True	38.0	61000.0	0
4	False	True	False	40.0	63778.0	1
5	True	False	False	35.0	58000.0	1
6	False	False	True	38.0	52000.0	0
7	True	False	False	48.0	79000.0	1
8	False	True	False	50.0	83000.0	0
9	True	False	False	37.0	67000.0	1

Next steps: Generate code with updated_dataset | New interactive sheet

Start coding or generate with AI.

Variables Terminal 5:55 PM Python 3

Commands + Code + Text Run all

RAM Disk

```
#Experiment-5(to detect outliers in a given data set)
#yashwanth kumar v
#240701609
```

```
import numpy as np
array=np.random.randint(1,100,16)
array
```

```
array([32, 93, 10, 75, 19, 18, 88, 93, 27, 80, 23, 17, 63, 67, 12, 30])
```

```
array.mean()
```

```
np.float64(46.6875)
```

```
np.percentile(array,25)
```

```
np.float64(18.75)
```

```
np.percentile(array,50)
```

```
np.float64(31.0)
```

```
np.percentile(array,75)
```

```
np.float64(76.25)
```

```
np.percentile(array,100)
```

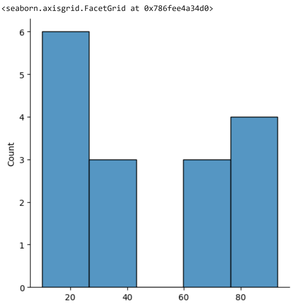
```
np.float64(93.0)
```

```
def outDetection(array):
  sorted(array)
  Q1,Q3=np.percentile(array,[25,75])
  IQR=Q3-Q1
  lr=Q1-(1.5*IQR)
  ur=Q3+(1.5*IQR)
  return lr,ur
lr,ur=outDetection(array)
lr,ur
```

```
(np.float64(-67.5), np.float64(162.5))
```

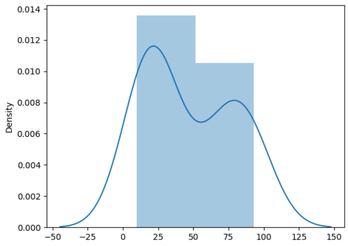
```
import seaborn as sns
%matplotlib inline
sns.displot(array)
```

```
<seaborn.axisgrid.FacetGrid at 0x786fee4a34d0>
```




```
sns.distplot(array)
```

```
/tmp/ipython-input-1133588802.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(array)
<Axes: ylabel='Density'>
```

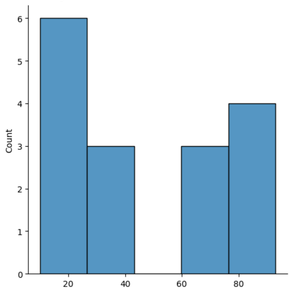



```
new_array=array[(array>lr) & (array<ur)]
new_array
```

```
array([32, 93, 10, 75, 19, 18, 88, 93, 27, 80, 23, 17, 63, 67, 12, 30])
```

```
sns.displot(new_array)
```

```
<seaborn.axisgrid.FacetGrid at 0x786fb9f63d70>
```




```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

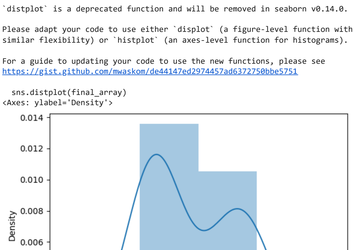
```
(np.float64(-67.5), np.float64(162.5))
```

```
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
```

```
array([32, 93, 10, 75, 19, 18, 88, 93, 27, 80, 23, 17, 63, 67, 12, 30])
```

```
sns.distplot(final_array)
```

```
/tmp/ipython-input-209491988.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(final_array)
<Axes: ylabel='Density'>
```




```
Start coding or generate with AI.
```

Variables Terminal 6:01 PM Python 3

Commands + Code + Text Run all

RAM Disk

[1] 0s
```
#Experiment-6(Experiment to understand feature scaling)
#yashwanth kumar v
#240701609
```

[10] 0s
```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/pre_process_datasample.csv")
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

Next steps: Generate code with df New interactive sheet

[11] 0s
```
df.head()
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes

Next steps: Generate code with df New interactive sheet

[12] 0s
```
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
```
```
/tmp/ipython-input-3424832005.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df.Country.fillna(df.Country.mode()[0],inplace=True)
```

[13] 0s
```
label=df.iloc[:,-1].values
```

[15] 4s
```
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
```
```
SimpleImputer()
```

[16] 0s
```
Salary.fit(features[:,[2]])
```
```
SimpleImputer()
```

[17] 0s
```
SimpleImputer()
```
```
SimpleImputer()
```

[18] 0s
```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```
```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['Germany', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

[19] 0s
```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```
```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

[20] 0s
```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```
```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 1.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

[21] 0s
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```
```
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

[22] 0s
```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```
```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [0.        , 1.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Variables Terminal 6:13 PM Python 3

Commands + Code + Text ▷ Run all Saving failed since 10:04 PM

```
#Experiment-7(EDA)
#yashwanth kumar v
#240701609
```

[1]

```python
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

[2]

```python
tips = sns.load_dataset('tips')
tips.head()
```

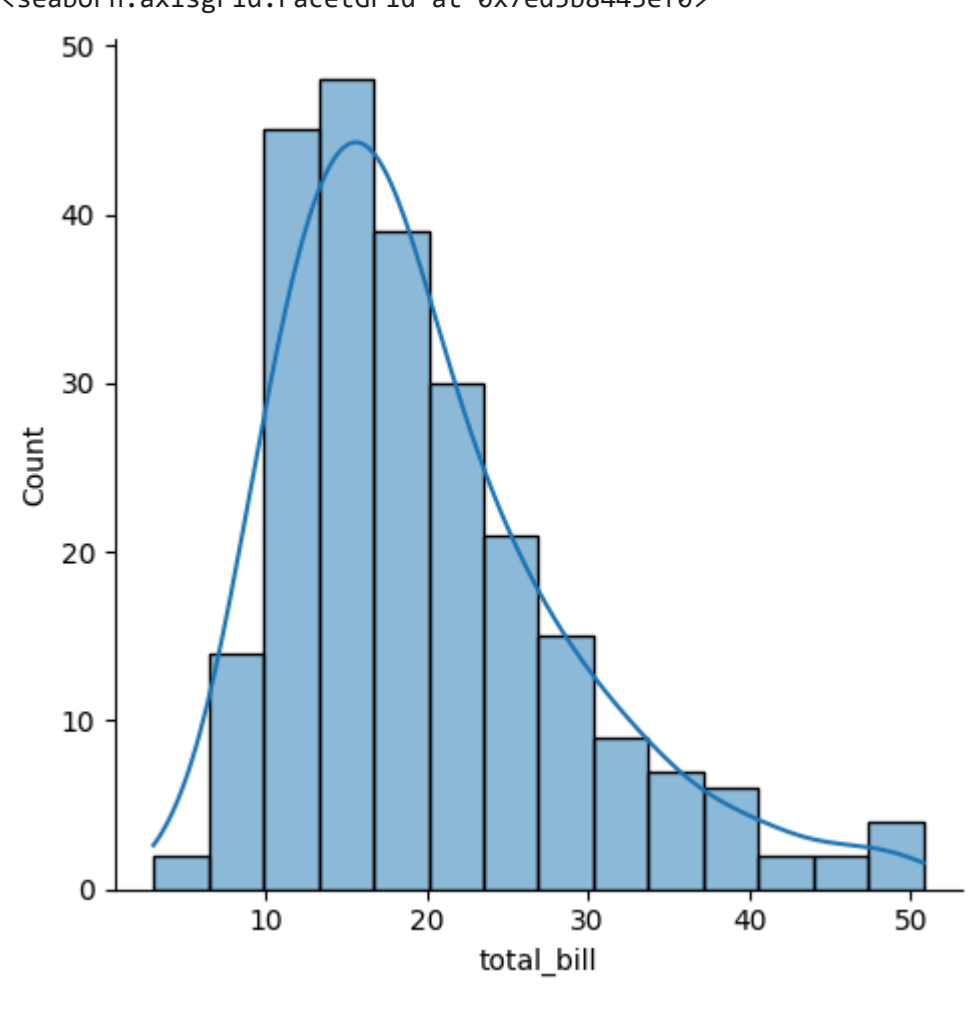
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Next steps: Generate code with tips New interactive sheet

[3]

```python
sns.displot(tips['total_bill'], kde=True)
```

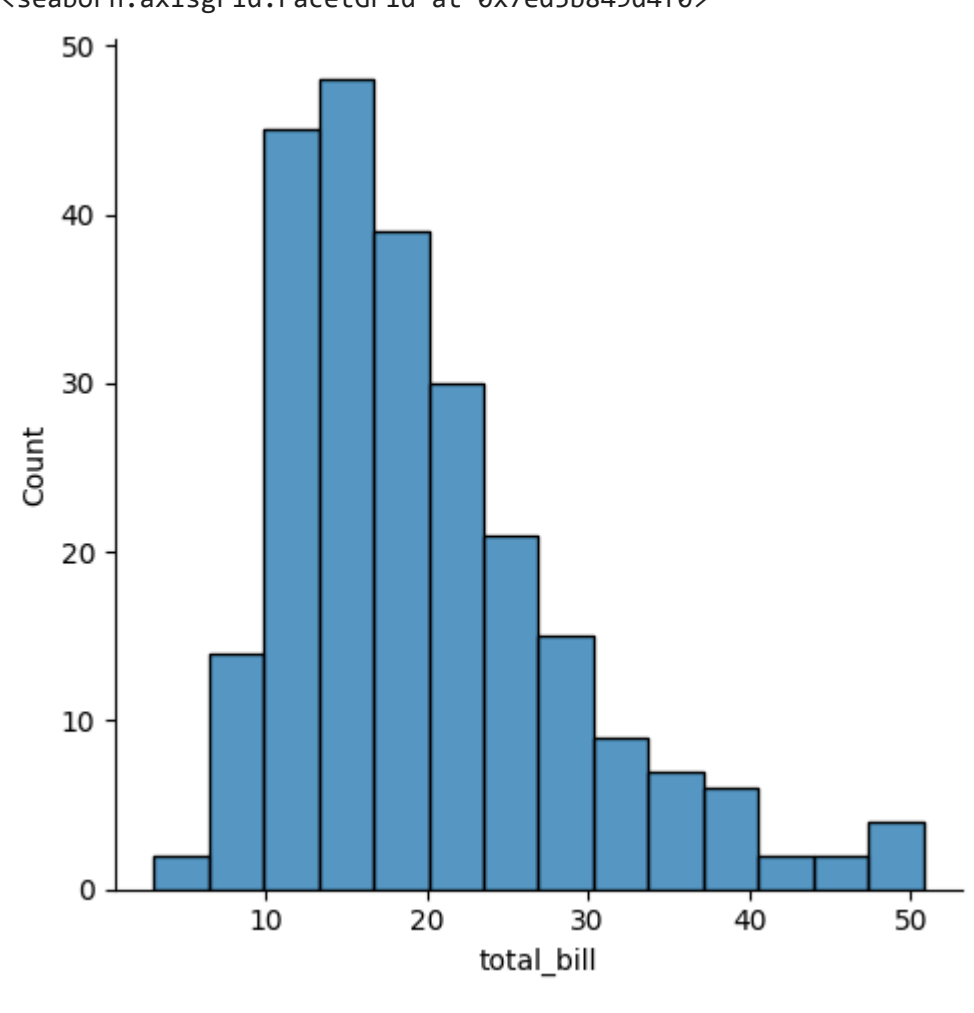
```
<seaborn.axisgrid.FacetGrid at 0x7ed5b8443ef0>
```



[4]

```python
sns.displot(tips['total_bill'], kde=False)
```

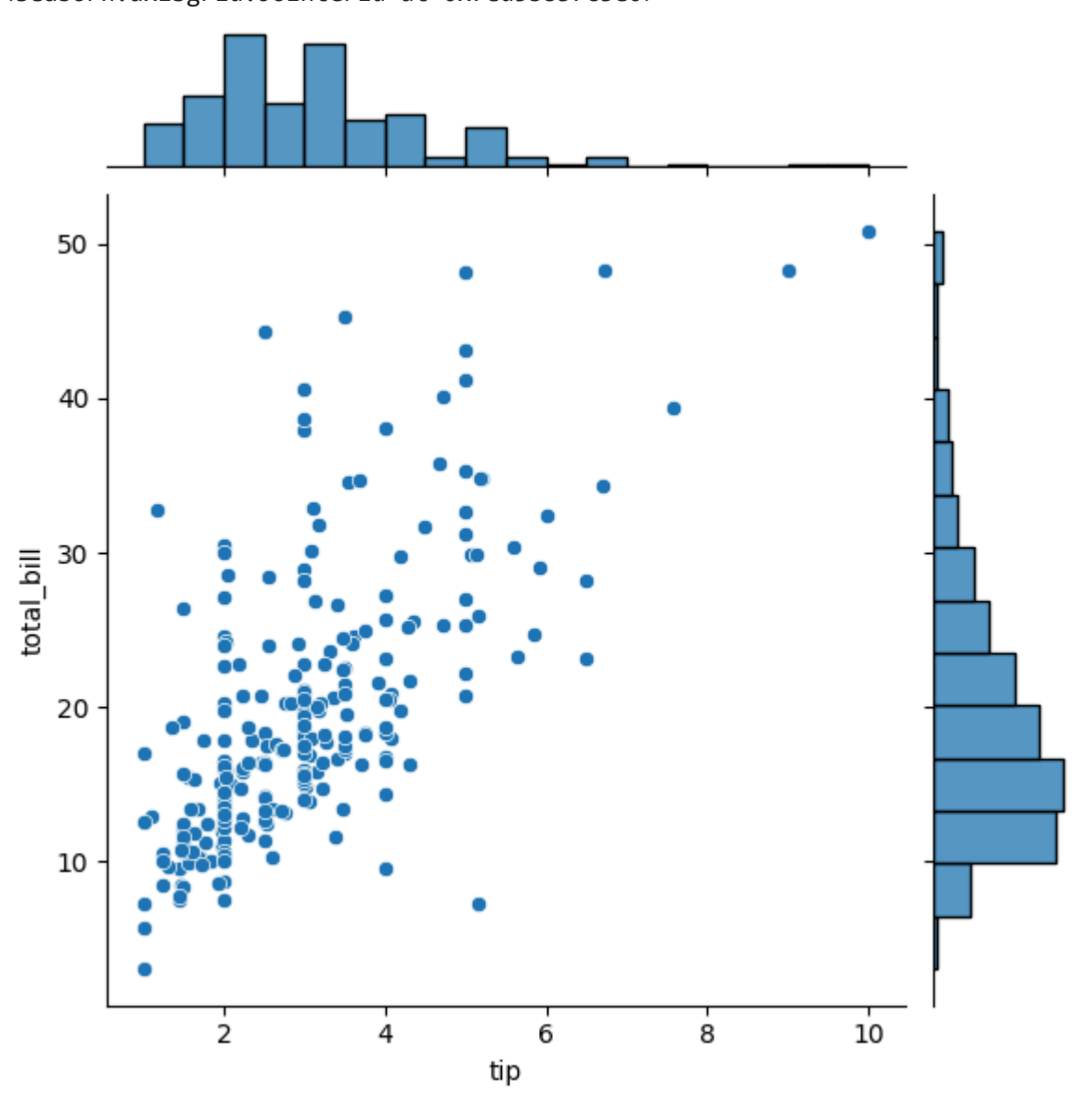
```
<seaborn.axisgrid.FacetGrid at 0x7ed5b849d4f0>
```



[5]

```python
sns.jointplot(x=tips['tip'], y=tips['total_bill'])
```

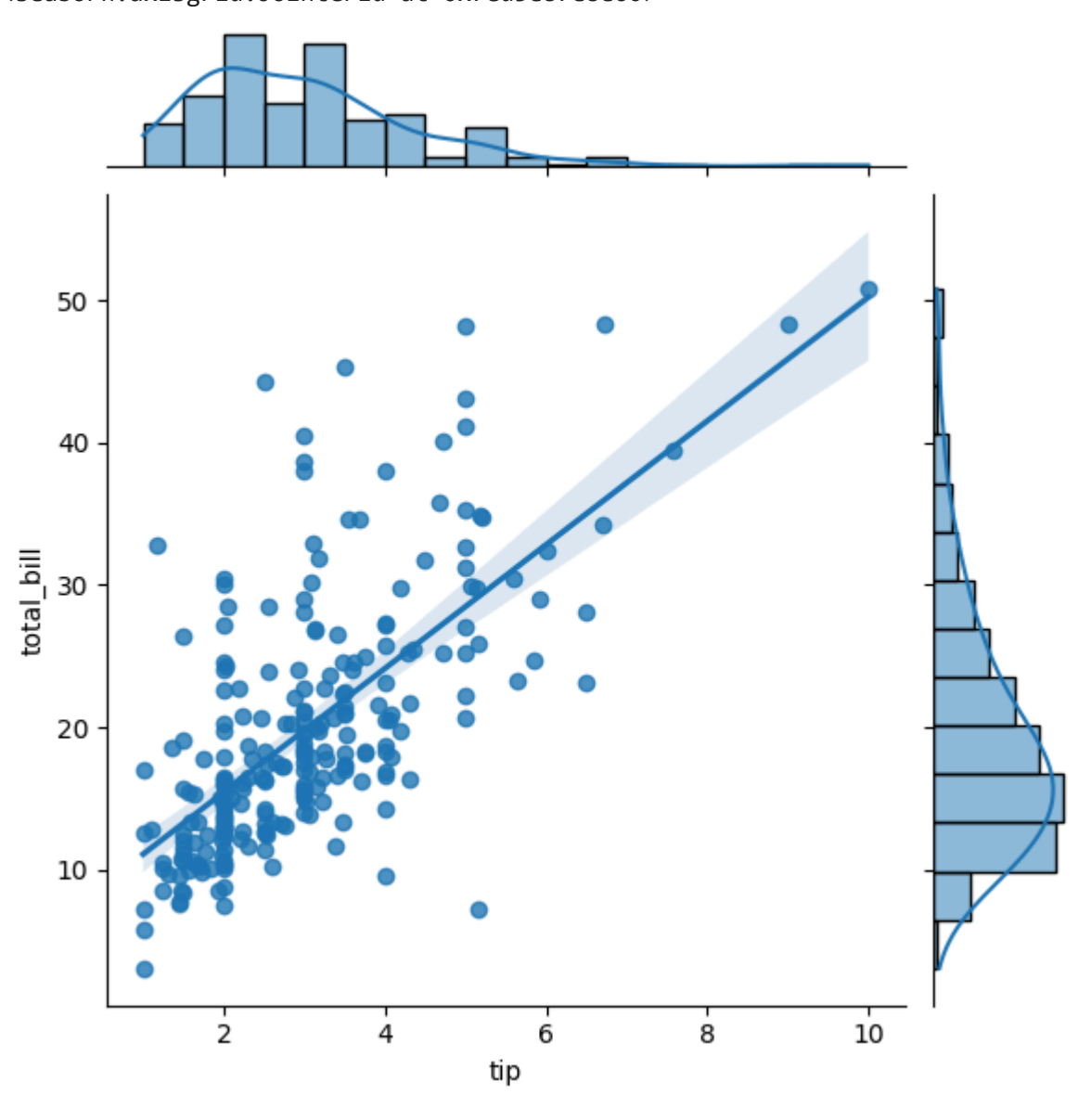
```
<seaborn.axisgrid.JointGrid at 0x7ed5b837c3e0>
```



[6]

```python
sns.jointplot(x=tips['tip'], y=tips['total_bill'], kind='reg')
```

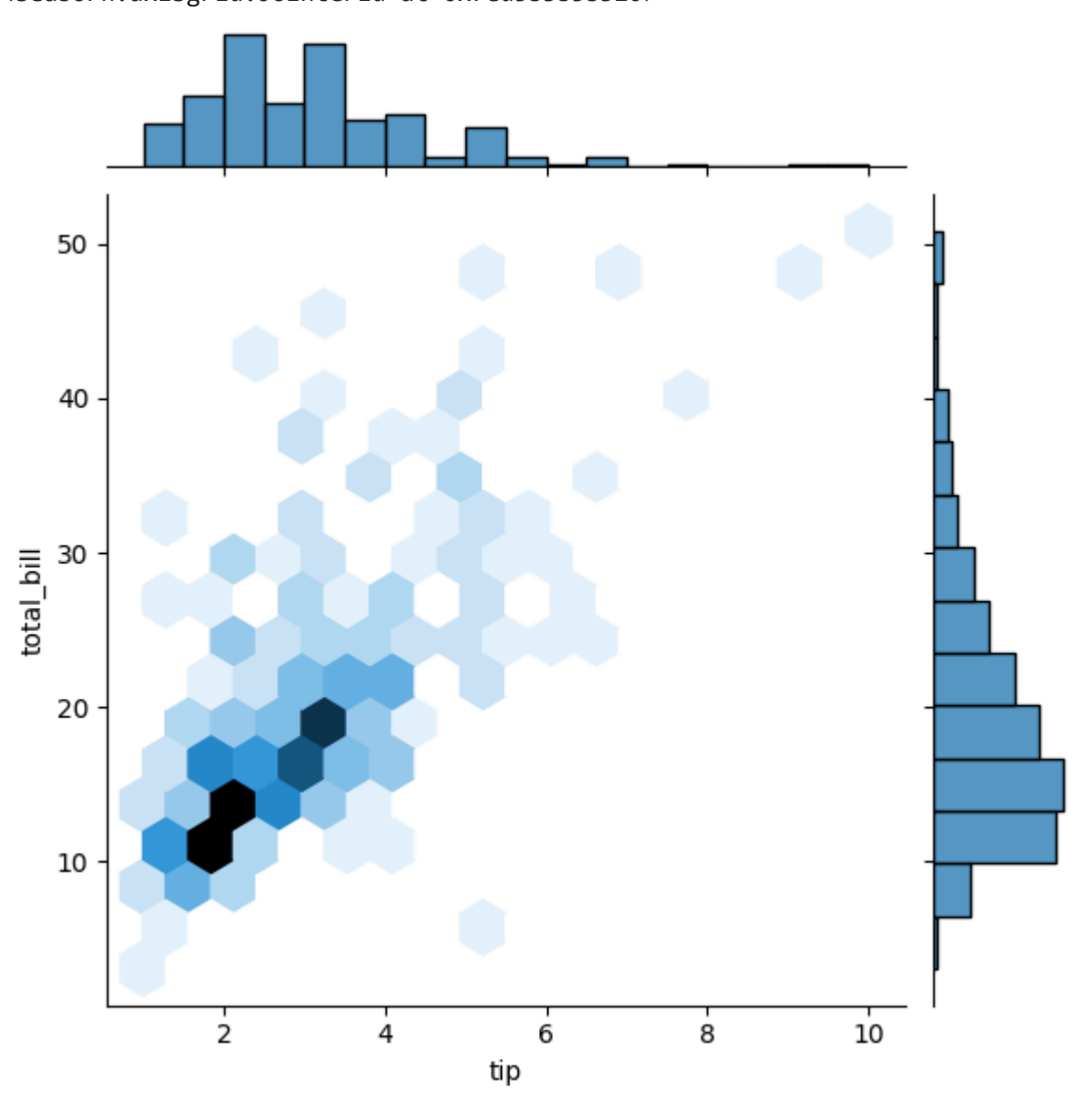
```
<seaborn.axisgrid.JointGrid at 0x7ed5e87e8e60>
```



[7]

```python
sns.jointplot(x=tips['tip'], y=tips['total_bill'], kind='hex')
```

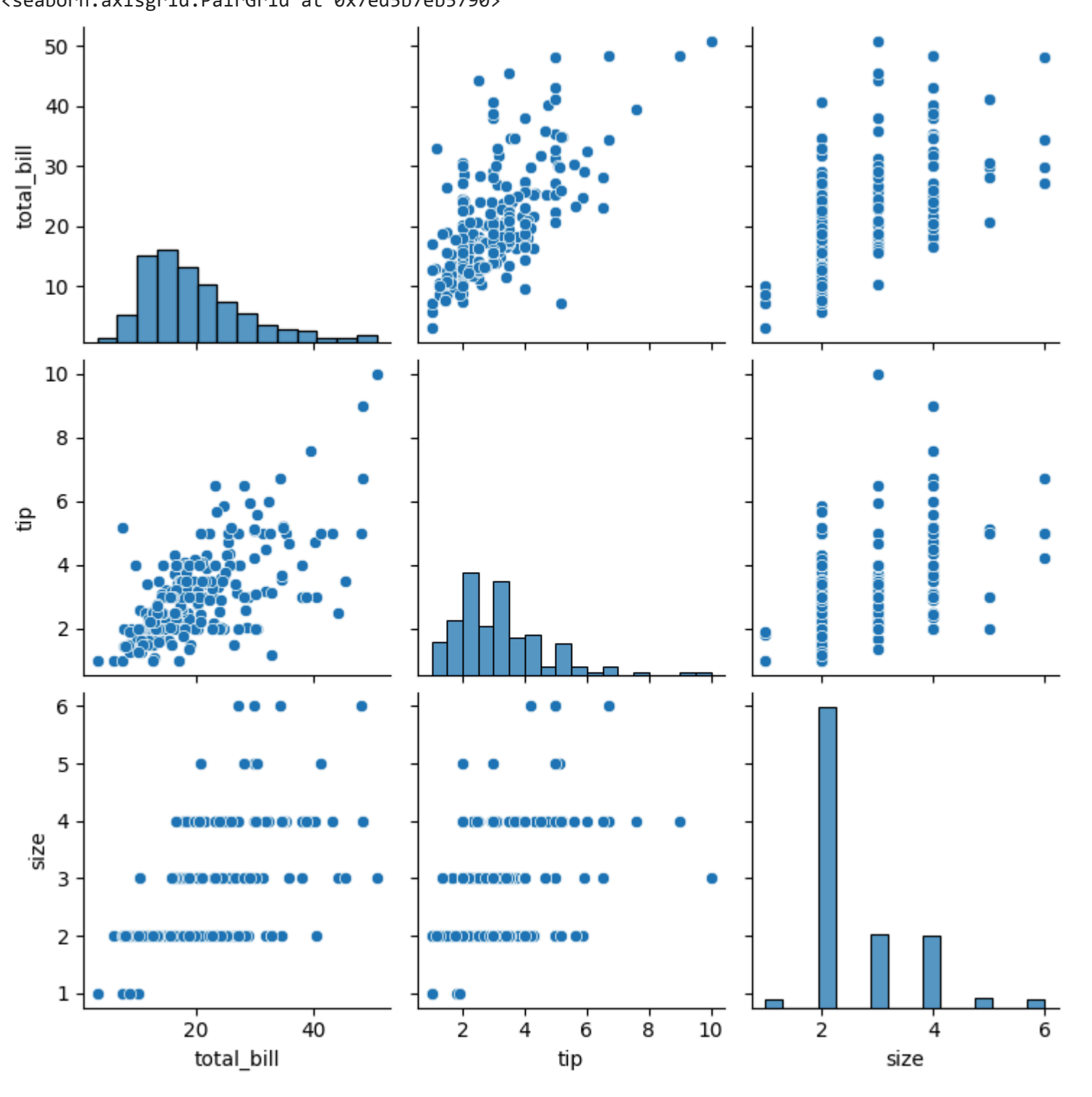
```
<seaborn.axisgrid.JointGrid at 0x7ed5b3b5b320>
```



[8]

```python
sns.pairplot(tips)
```

```
<seaborn.axisgrid.PairGrid at 0x7ed5b7eb5790>
```



[9]

```python
tips.time.value_counts()
```

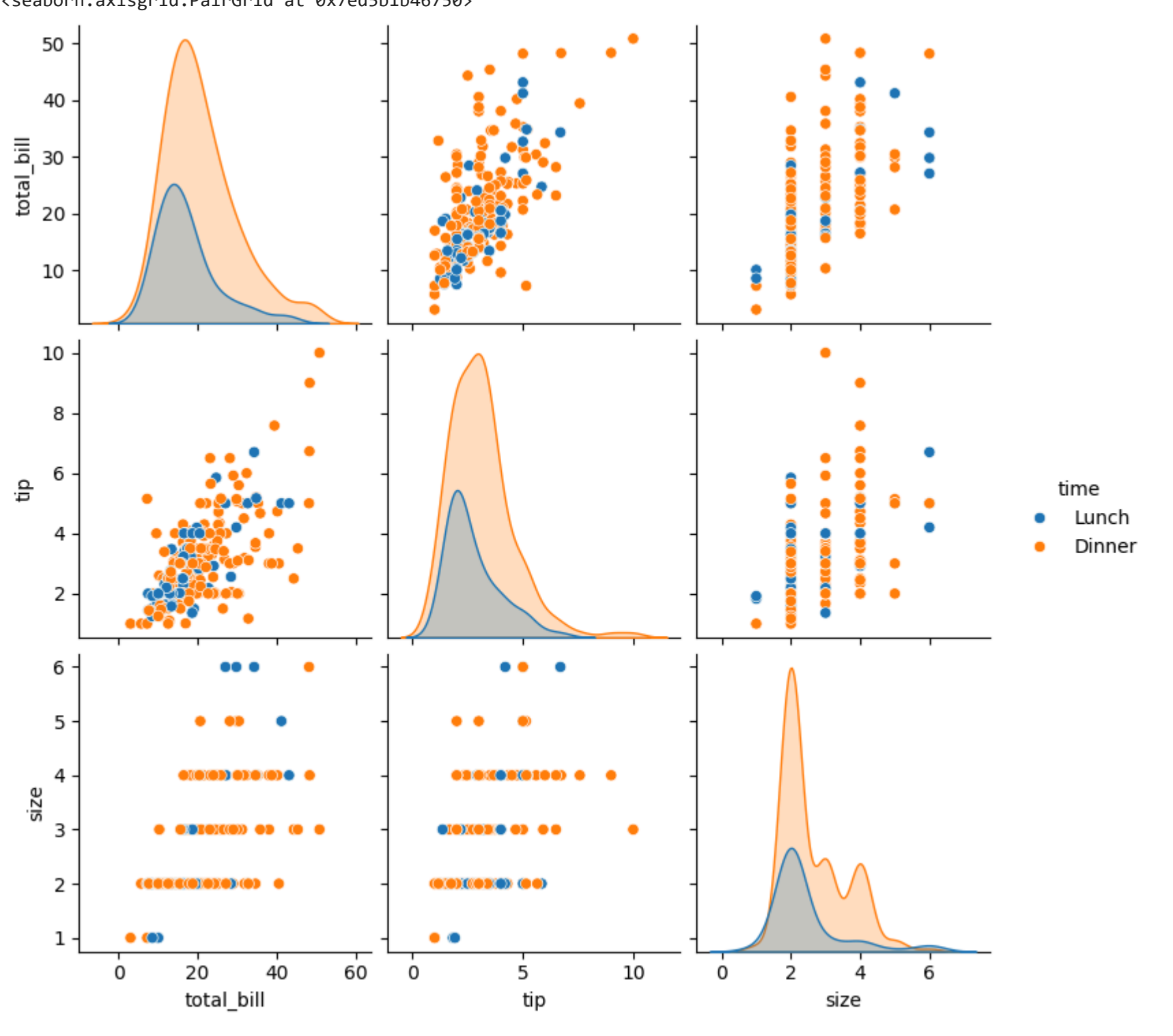
time	count
Dinner	176
Lunch	68

dtype: int64

[10]

```python
sns.pairplot(tips, hue='time')
```

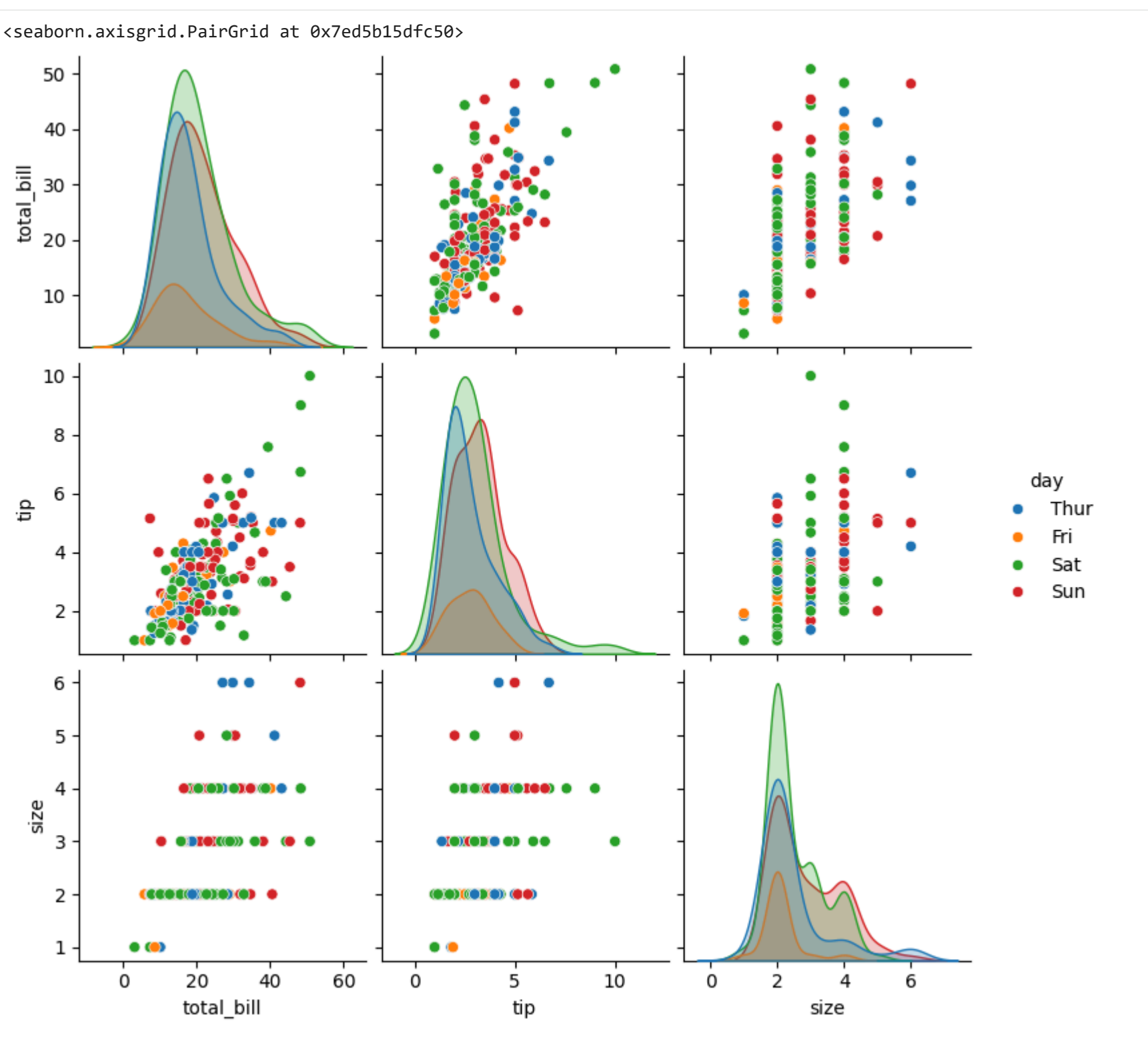
```
<seaborn.axisgrid.PairGrid at 0x7ed5b1b46750>
```



[11]

```python
sns.pairplot(tips, hue='day')
```

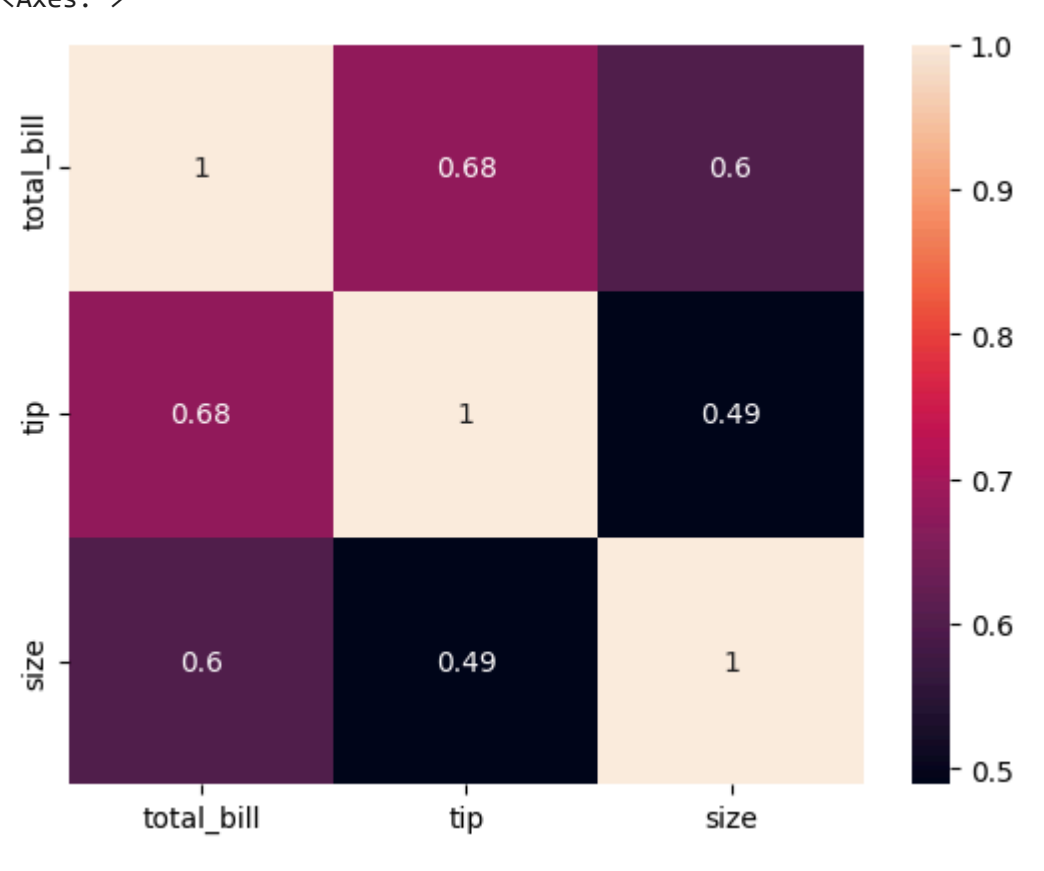
```
<seaborn.axisgrid.PairGrid at 0x7ed5b15dfc50>
```



[12]

```python
sns.heatmap(tips.corr(numeric_only=True), annot=True)
```

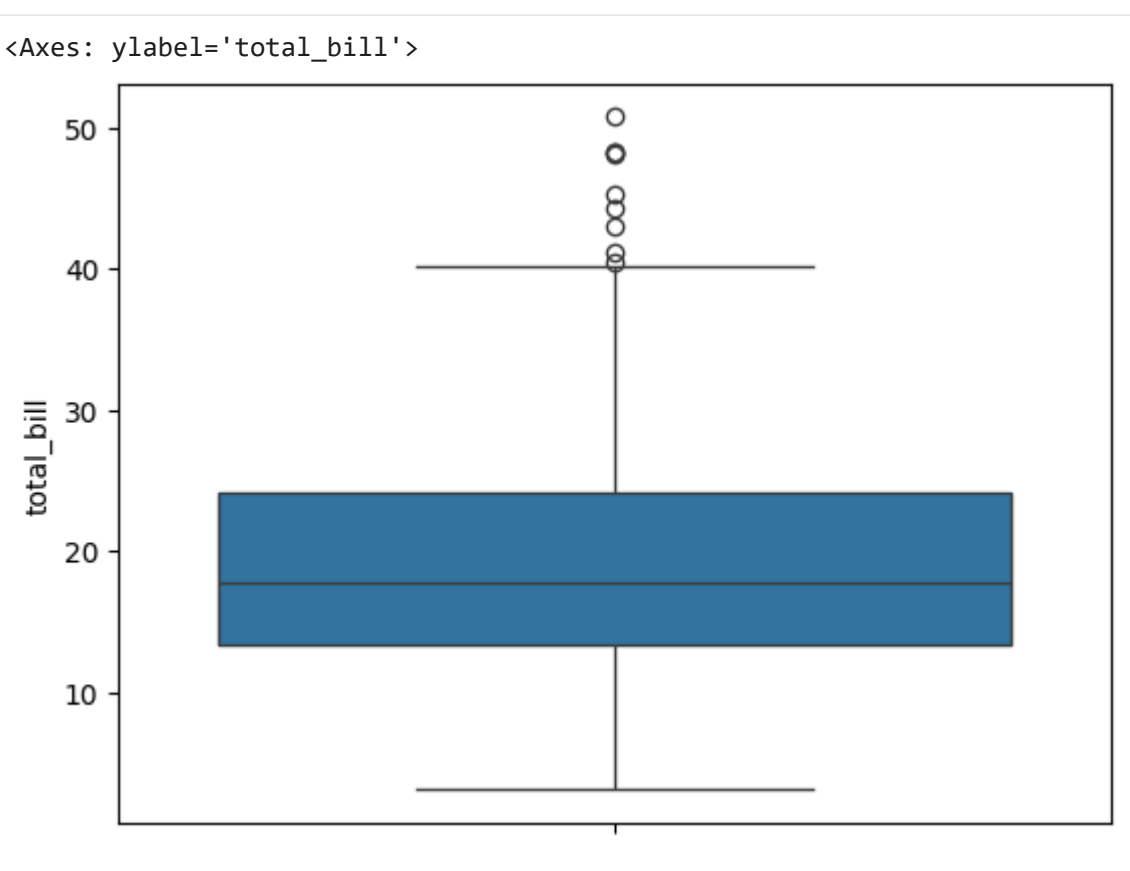
```
<Axes: >
```



[13]

```python
sns.boxplot(tips['total_bill'])
```

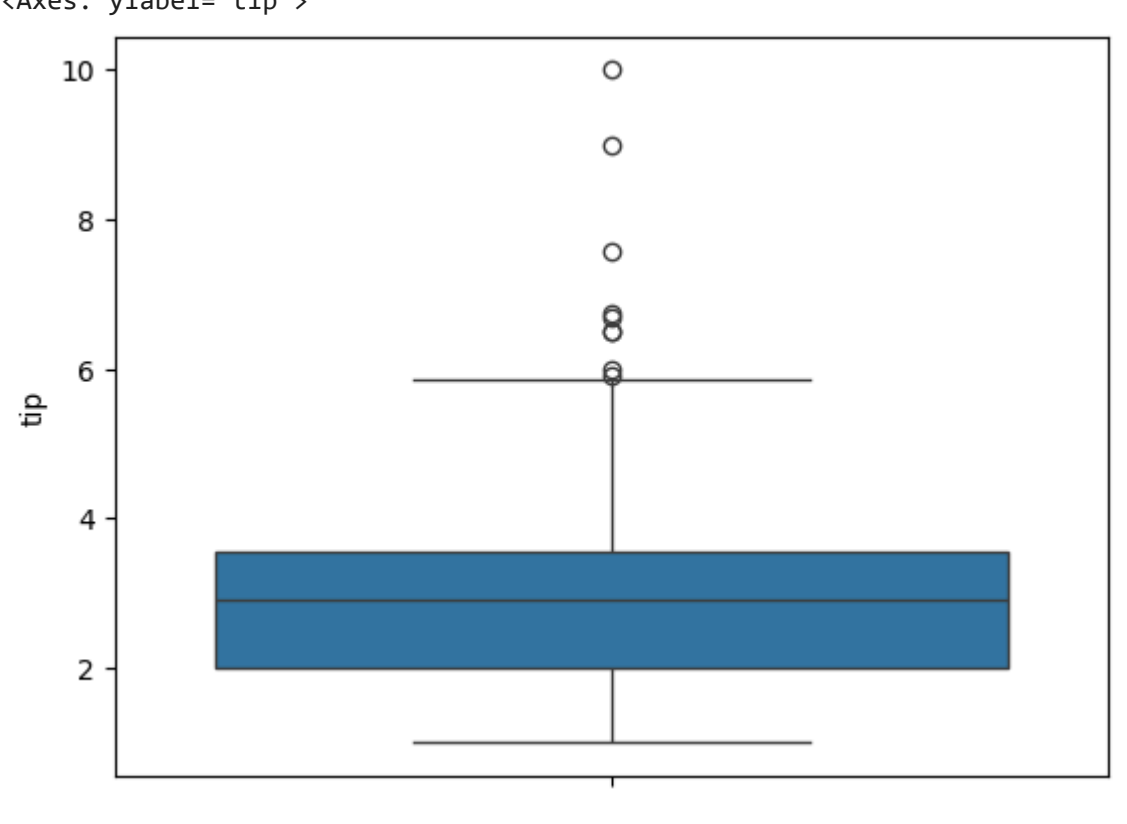
```
<Axes: ylabel='total_bill'>
```



[14]

```python
sns.boxplot(tips['tip'])
```

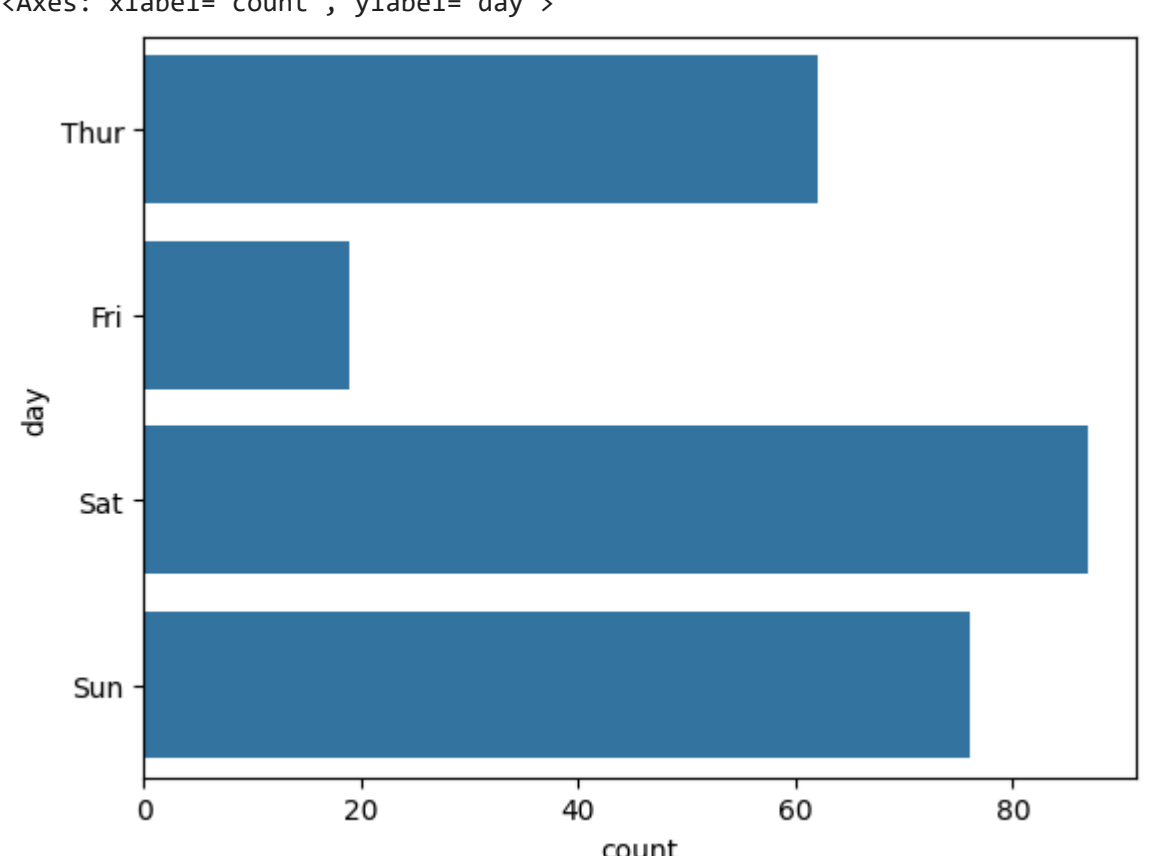
```
<Axes: ylabel='tip'>
```



[15]

```python
sns.countplot(tips['day'])
```

```
<Axes: xlabel='count', ylabel='day'>
```



Automatic saving failed. This file was updated remotely or in another tab. Show diff

✓ 10:06 PM Python 3

Commands + Code + Text Run all

RAM Disk

```
#Experiment-8(linear regression)
#yashwanth kumar v
#240701609
```

```
import numpy as np
import pandas as pd
```

```
# Create sample salary dataset
data = {
    'YearsExperience': [1.1, 1.3, 1.5, 2.0, 2.2, 2.9, 3.0, 3.2, 3.2, 3.7, 3.9, 4.0, 4.0, 4.1, 4.5, 4.9, 5.1, 5.3, 5.9, 6.0, 6.8, 7.1, 7.9, 8.2, 8.7, 9.0, 9.5, 9.6, 10.3, 10
    'Salary': [37731, 43525, 39891, 56642, 60150, 54445, 64445, 57189, 63218, 55794, 56957, 57081, 61111, 67938, 66029, 83088, 81363, 93940, 91738, 98273, 101302, 113812, 1
}
df = pd.DataFrame(data)
df
```

	YearsExperience	Salary
0	1.1	37731
1	1.3	43525
2	1.5	39891
3	2.0	56642
4	2.2	60150
5	2.9	54445
6	3.0	64445
7	3.2	57189
8	3.2	63218
9	3.7	55794
10	3.9	56957
11	4.0	57081
12	4.0	61111
13	4.1	67938
14	4.5	66029
15	4.9	83088
16	5.1	81363
17	5.3	93940
18	5.9	91738
19	6.0	98273
20	6.8	101302
21	7.1	113812
22	7.9	109431
23	8.2	105582
24	8.7	116969
25	9.0	112635
26	9.5	122391
27	9.6	121872
28	10.3	91738
29	10.5	93940

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	79340.666667
std	2.837888	26128.480351
min	1.100000	37731.000000
25%	3.200000	57108.000000
50%	4.700000	74650.500000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
features = df.iloc[:, 0].values
label = df.iloc[:, 1].values
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=0)
```

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train.reshape(-1, 1), y_train)
```

```
▾ LinearRegression i ?
LinearRegression()
```

```
model.score(x_train.reshape(-1, 1), y_train)
```

```
0.8308613552280183
```

```
model.score(x_test.reshape(-1, 1), y_test)
```

```
0.7657841556832163
```

```
model.coef_
```

```
array([8575.37154813])
```

```
model.intercept_
```

```
np.float64(35136.91225239668)
```

```
import pickle
pickle.dump(model, open('SalaryPred.model', 'wb'))
```

```
model = pickle.load(open('SalaryPred.model', 'rb'))
yrof_exp = float(input('Enter Years of Experience '))
yrof_expNP = np.array(yrof_exp)
Salary = model.predict(yrof_expNP.reshape(-1, 1))
print('Estimated Salary for {} years of experience is {}'.format(yrof_exp, Salary))
```

```
Enter Years of Experience 5
Estimated Salary for 5.0 years of experience is [78013.76999303]
```

Variables Terminal Python 3

Files

..
sample_data
Social_Network_Ads.csv

[ ]
```
#Experiment-9(Logistic regression)
#yashwanth kumar v
#240701609
```

[9] ✓ 0s
```
import numpy as np
import pandas as pd
```

[10] ✓ 0s
```
df = pd.read_csv('/content/Social_Network_Ads.csv')
df
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

Next steps: Generate code with df New interactive sheet

[11] ✓ 0s
```
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Next steps: Generate code with df New interactive sheet

[12] ✓ 0s
```
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values
```

[13] ✓ 0s
```
features
```
Show hidden output

[14] ✓ 0s
```
label
```
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

[15] ✓ 1s
```
from sklearn.model_selection import train_test_split
```

[16] ✓ 0s
```
from sklearn.linear_model import LogisticRegression
```

[17] ✓ 2s
```
# Find best random_state
for i in range(1, 40):
    x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=i)
    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print('Test {} Train {} Random State {}'.format(test_score, train_score, i))
```
```
Test 0.9 Train 0.840625 Random State 4
Test 0.8625 Train 0.85 Random State 5
Test 0.8625 Train 0.859375 Random State 6
Test 0.8875 Train 0.8375 Random State 7
Test 0.8625 Train 0.8375 Random State 9
Test 0.9 Train 0.840625 Random State 10
Test 0.8625 Train 0.85625 Random State 14
Test 0.85 Train 0.84375 Random State 15
Test 0.8625 Train 0.85625 Random State 16
Test 0.875 Train 0.834375 Random State 18
Test 0.85 Train 0.84375 Random State 19
Test 0.875 Train 0.84375 Random State 20
Test 0.8625 Train 0.834375 Random State 21
Test 0.875 Train 0.840625 Random State 22
Test 0.875 Train 0.840625 Random State 24
Test 0.85 Train 0.834375 Random State 26
Test 0.85 Train 0.840625 Random State 27
Test 0.8625 Train 0.834375 Random State 30
Test 0.8625 Train 0.85625 Random State 31
Test 0.875 Train 0.853125 Random State 32
Test 0.8625 Train 0.84375 Random State 33
Test 0.875 Train 0.83125 Random State 35
Test 0.8625 Train 0.853125 Random State 36
Test 0.8875 Train 0.840625 Random State 38
Test 0.875 Train 0.8375 Random State 39
```

[18] ✓ 0s
```
# Train final model with best random_state
x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=25)
final_model = LogisticRegression()
final_model.fit(x_train, y_train)
print(final_model.score(x_train, y_train))
print(final_model.score(x_test, y_test))
```
```
0.846875
0.8
```

[19] ✓ 0s
```
from sklearn.metrics import classification_report
print(classification_report(label, final_model.predict(features)))
```
```
              precision    recall  f1-score   support

           0       0.84      0.92      0.88       257
           1       0.82      0.69      0.75       143

    accuracy                           0.84       400
   macro avg       0.83      0.81      0.82       400
weighted avg       0.84      0.84      0.83       400
```

[ ] Start coding or generate with AI.

How can I install Python libraries? Load data from Google Drive Show an example of training a
Colab paid products - Cancel contracts here

What can I help you build?

Disk 69.58 GB available

Automatic saving failed. This file was updated remotely or in another tab. Show diff

5:10 AM Python 3

Files
.
sample_data
Mall_Customers.csv

```
#Experiment-10(K-Means Clustering)
#yashwanth kumar v
#240701609
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
df = pd.read_csv('/content/Mall_Customers.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

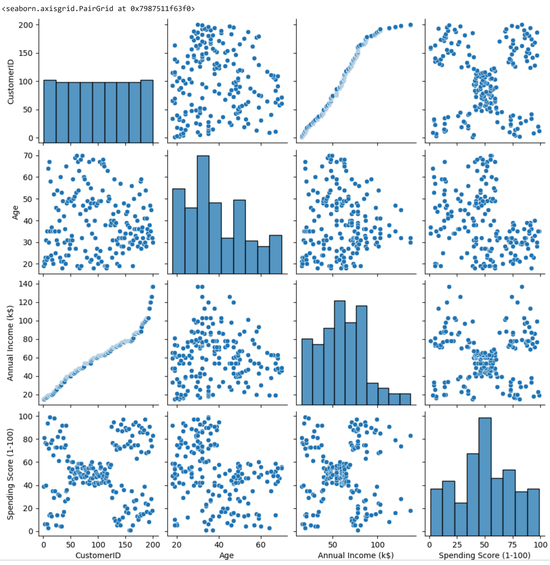
```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with df New interactive sheet

```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x7987511f63f0>
```




```
features = df.iloc[:, [3, 4]].values
```

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=5)
model.fit(features)
```

```
KMeans(n_clusters=5)
```

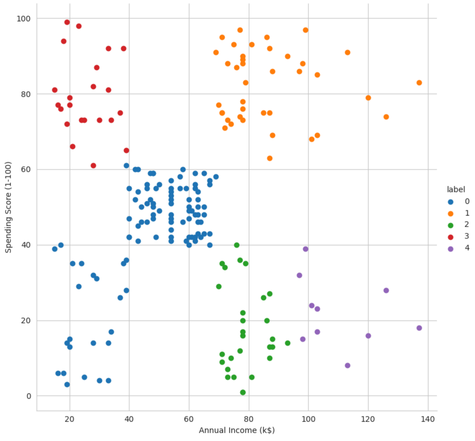
```
Final = df.iloc[:, [3, 4]]
Final['label'] = model.predict(features)
Final.head()
```

```
/tmp/ipython-input-1374593914.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  Final['label'] = model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	0
1	15	81	3
2	16	6	0
3	16	77	3
4	17	40	0

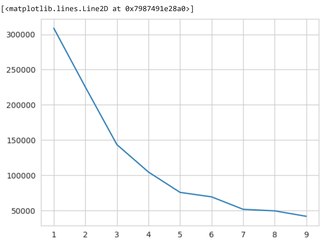
Next steps: Generate code with Final New interactive sheet

```
sns.set_style('whitegrid')
sns.FacetGrid(Final, hue='label', height=8).map(plt.scatter, 'Annual Income (k$)', 'Spending Score (1-100)').add_legend()
plt.show()
```




```
features_e1 = df.iloc[:, [2, 3, 4]].values
from sklearn.cluster import KMeans
wcss = []
for i in range(1, 10):
    model = KMeans(n_clusters=i)
    model.fit(features_e1)
    wcss.append(model.inertia_)
plt.plot(range(1, 10), wcss)
```

```
[<matplotlib.lines.Line2D at 0x7987491e28a0>]
```



```
Start coding or generate with AI.
```

Disk 69.57 GB available

Automatic saving failed. This file was updated remotely or in another tab. Show diff

5:20 AM Python 3

[ ]
```
#Experiment-11(KNN)
#yashwanth kumar v
#240701609
```

[29]
```
import numpy as np
import pandas as pd
```

[30]
```
df = pd.read_csv('/content/Iris (1).csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

[31]
```
df.variety.value_counts()
```

variety	count
Setosa	50
Versicolor	50
Virginica	50

dtype: int64

[32]
```
df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

Next steps: Generate code with df | New interactive sheet

[33]
```
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values
```

[34]
```
from sklearn.model_selection import train_test_split
```

[35]
```
from sklearn.neighbors import KNeighborsClassifier
```

[36]
```
x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=25)
modelKNN = KNeighborsClassifier(n_neighbors=5)
modelKNN.fit(x_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

[37]
```
print(modelKNN.score(x_train, y_train))
print(modelKNN.score(x_test, y_test))
```

```
0.9833333333333333
0.9333333333333333
```

[38]
```
from sklearn.metrics import confusion_matrix
confusion_matrix(label, modelKNN.predict(features))
```

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

[39]
```
from sklearn.metrics import classification_report
print(classification_report(label, modelKNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

[ ]
Start coding or generate with AI.

Files

..
sample_data
Iris (1).csv

[ ]
```
#Yashwanth kumar v
#240701609
```

[40]
```
# Experiment-12(T-TEST)
# Testing whether average marks = 70 at 5% significance level

import numpy as np
from scipy import stats

# Sample data
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])

# Hypothesized mean
mu0 = 70

# One-sample t-test
t_stat, p_value = stats.ttest_1samp(marks, mu0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis: Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis: No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis: No significant difference.
```

[41]
```
#Experiment -13 Z-TEST
# Testing manufacturer's claim that average weight = 50g

import numpy as np
from math import sqrt
from scipy.stats import norm

# Given data
x_bar = 51.2  # sample mean
mu0 = 50  # population mean
sigma = 3  # population standard deviation
n = 36  # sample size

# Calculate Z-statistic
z_stat = (x_bar - mu0) / (sigma / sqrt(n))

# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis: Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis: No significant difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis: Mean is significantly different from 50 g.
```

[42]
```
# Experiment-14 ANOVA TEST
# Testing if there's significant difference among three fertilizers (A, B, C)

import numpy as np
from scipy import stats

# Data for three fertilizers
A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis: Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis: No significant difference.")
```

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis: Means are significantly different.
```

[ ] Start coding or generate with AI.

Disk 69.57 GB available

Automatic saving failed. This file was updated remotely or in another tab. Show diff